Lecture-5

Building an HTTP Server in Node.js

From "Hello World" to serving real pages with routing



Basic Server



Serving a File



Routing Pages

TODAY

Three steps to a real Node.js server

1



Basic Server

Respond with plain text to every request

2



Serve a File

Read an HTML file from disk and send it back

3



Route Requests

Send a different page depending on the URL

STEP 1

The simplest possible server

```
const http = require('http');

const hostname = '127.0.0.1';

const port = 3000;

const server = http.createServer(     (req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World');
}                                    );

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

STEP 1

What each part does

`statusCode = 200`	Tells the browser: request succeeded
`setHeader(...)`	Tells the browser what kind of content is coming — here, plain text
`res.end(...)`	Sends the response body and closes the connection
`.listen(port, hostname)`	Starts the server, waiting for requests on 127.0.0.1:3000

STEP 2

Serving a real HTML file

```
const http = require('http');
const fs = require('fs');

const fileContent = fs.readFileSync('test.html');

const server = http.createServer(    (req, res) => {
  res.writeHead(200, { 'Content-type': 'text/html' });
  res.end(fileContent);
}.
                                 );

server.listen(80, '127.0.0.1', () => {
  console.log("Listening on port 80");
});
```

STEP 2

What changed from Step 1

`fs.readFileSync('test.html')`	Reads the HTML file from disk into memory — once, when the server starts
`writeHead(200, {...})`	Sets status + header together in a single call
`'Content-type': 'text/html'`	Tells the browser to render this as a webpage, not plain text
`res.end(fileContent)`	Sends the actual HTML file back as the response

STEP 3

One server, many pages



The problem

So far, every request gets the SAME file back. A real site needs a Home page, an About page, a Contact page — all from one server.

The fix: check req.url

Every request carries the path the browser asked for — /, /about, /contact. We read req.url and decide which file to send back.

```
/  →  home.html          /about  →  about.html          /contact  →  contact.html
```

STEP 3

Load every page up front

```
const http = require('http');
const fs = require('fs');

const home    = fs.readFileSync('index.html');
const about   = fs.readFileSync('./about.html');
const music   = fs.readFileSync('./music.html');
const contact = fs.readFileSync('./contact.html');
```

Each HTML file is read once, when the server starts, and kept ready in memory — so every request can be answered instantly without touching the disk again.

STEP 3

Choosing a page with req.url

```
const server = http.createServer((req, res) => {
  const url = req.url;
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/html');

  if (url == '/') { res.end(home); }
  else if (url == '/about') { res.end(about); }
  else if (url == '/music') { res.end(music); }
  else if (url == '/contact') { res.end(contact); }
  else {
    res.statusCode = 404;
    res.end("<h1>404 not found</h1>");
  }
});
```

STEP 3

Always handle the unknown route



Why the final else matters

If the URL doesn't match any known route, the server must still respond — otherwise the request hangs forever.

`statusCode = 404` tells the browser: page not found.
`res.end("<h1>404 not found</h1>")` sends a simple error page as the response.

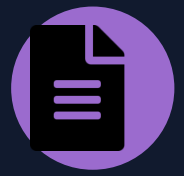
RECAP

From one response to a real website



Step 1: Basic Server

createServer + listen — always respond



Step 2: Serve a File

readFileSync + writeHead to send real HTML



Step 3: Route by URL

req.url decides which page to send



Always add a 404

Handle unknown routes so requests never hang

Thank You !